Project Title :

Smart Hospital Management System

Name: Kethineni Venkata Avinash

Registration Number: 25BAI10364

School: SCAI

Branch: B.Tech Computer Science and Engineering (Artificial Intelligence & Machine Learning)

Year: First Year

Course Code: CSA2001 - Fundamentals in AI and ML

Date: March 31, 2026

Executive Summary

This project aims to solve critical bottlenecks in modern urban healthcare systems by developing a unified, AI-driven Smart Hospital Management System. The application seamlessly integrates multiple subfields of Artificial Intelligence—including Symbolic AI (Expert Systems), Graph Search Algorithms, Machine Learning, Computer Vision, and Natural Language Processing (NLP) into a single, user-friendly Streamlit web application. By automating triage, ambulance routing, recovery prediction, and feedback analysis, this system demonstrates a comprehensive understanding of the Fundamentals in AI and ML.

Introduction

The healthcare sector is a highly complex environment where rapid decision-making directly impacts patient outcomes. The Smart Hospital Management System is an integrated, AI-driven application designed to streamline the critical path of patient care from initial symptom triage and emergency dispatch to recovery prediction and post-discharge feedback. This project serves as a capstone application, bridging Symbolic AI, Graph Search Algorithms, and Statistical Machine Learning into a cohesive, interactive web application.

Problem Statement

In emergency medical scenarios, time and resource allocation are critical. When a patient reports symptoms, hospitals face several immediate challenges:

1. Quickly diagnosing the condition and assessing the emergency risk level without immediate human physician availability.
2. Locating the nearest hospital facility and dispatching an ambulance using the most optimal route through city traffic.
3. Estimating the patient's recovery time to manage bed availability and hospital resources.
4. Processing unstructured patient feedback efficiently to improve service quality.

I chose to build this BYOP project because there is no single "out-of-the-box" algorithm that solves all these problems simultaneously. It requires designing a pipeline where deterministic logic rules, heuristic searches, and probabilistic machine learning models work together.

Objectives

- **Automated Triage:** Develop a rule-based expert system to diagnose diseases and assign risk levels based on overlapping symptoms.
- **Optimal Dispatch:** Implement optimal and complete search algorithms to find the fastest ambulance route to a patient.
- **Resource Forecasting:** Train a machine learning model to predict the number of recovery days required based on demographic and severity factors.
- **Administrative Automation:** Utilize contemporary AI techniques (Computer Vision and NLP) to automate patient check-ins and parse qualitative hospital feedback.

Functional Requirements

The system must successfully perform the following functions:

- Accept patient symptoms via a user interface and query a Prolog knowledge base to return the highest-probability diagnosis and risk level.
- Display a geospatial map of nearby hospital facilities.
- Simulate an urban grid and calculate the shortest path for an ambulance using both Breadth-First Search (BFS) and A* Search.
- Predict a continuous numerical value for "Recovery Days" using a Supervised Machine Learning model.
- Access a local camera feed to detect human faces in real-time.
- Classify raw text feedback into Positive, Neutral, or Negative sentiment categories with actionable suggestions.

Detailed Technical Architecture

The system is modularized into four core backend scripts, orchestrated by a central Streamlit frontend. Below is a detailed breakdown of each module and its alignment with the course syllabus.

Knowledge Representation & Expert System (medical_logic.pl)

- I utilized Prolog to build a declarative, rule-based expert system for medical triage.
- **Facts & Rules:** The knowledge base contains facts mapping 10 common and severe diseases (e.g., flu, dengue, covid19, heart attack) to their specific symptoms, treatments, and risk levels (low, moderate, high, critical).
- **Fuzzy Symptom Matching:** Real patients rarely present with a perfect textbook list of symptoms. To handle this, I designed a custom count_matches/3 recursive rule in Prolog. The best_diagnosis rule uses findall and setof predicates to evaluate the patient's inputted symptoms against the knowledge base, scoring and sorting diseases based on the highest symptom overlap (requiring a minimum match score of 2 to avoid false positives).
- **Dynamic Memory:** The system utilizes assertz and retract to dynamically register and discharge patients, generating unique Patient IDs during runtime.

Search Strategies for Ambulance Routing (search_engine.py)

- To simulate urban ambulance routing, I used Python's networkx library to generate a 10x10 city grid graph, strategically removing specific nodes to represent traffic obstacles or roadblocks.
- **Uninformed Search (BFS):** I implemented Breadth-First Search (nx.shortest_path) to find the path with the fewest number of edges. While it guarantees the shortest path in an unweighted grid, it explores nodes uniformly in all directions.
- **Informed Search (A Algorithm)**: I implemented the A* search algorithm (nx.astar_path) utilizing a Manhattan Distance heuristic ($|x_1 - x_2| + |y_1 - y_2|$). Because an ambulance traversing a grid cannot move diagonally, the Manhattan distance is an admissible heuristic (it never overestimates the true cost), guaranteeing an optimal path while exploring significantly fewer nodes than BFS.
- The system dynamically compares the execution time of both algorithms and selects the most efficient route, converting grid steps into real-world estimated travel times (assuming 40 km/h ambulance speeds).

Machine Learning for Prognosis (prognosis_ml.py)

- **Data Generation:** Due to medical data privacy constraints, I engineered a synthetic, statistically realistic dataset of 200 patients using NumPy. The dataset correlates patient age and disease severity (scaled 1-10 based on Prolog risk levels) to an estimated number of recovery days.
- **Supervised Learning:** I utilized Scikit-Learn to train a Multiple Linear Regression model. The features (X) are Age and Severity, and the target (Y) is Recovery Days.
- **Model Validation:** Following best practices taught in the course, the data is split into an 80/20 train/test ratio. The model is evaluated using Mean Squared Error (MSE) and the R^2 Score to ensure it does not suffer from severe underfitting or overfitting before predicting the actual user's recovery time.

Contemporary AI Integrations (advanced_features.py)

- **Computer Vision (Face Detection):** I implemented an automated patient check-in simulator using OpenCV. It accesses the local webcam and applies a pre-trained Haar Cascade Classifier (haarcascade_frontalface_default.xml) to detect patient faces in real-time, drawing bounding boxes and capturing a frame upon successful detection.
- **Natural Language Processing (Sentiment Analysis):** To handle post-visit hospital feedback, I integrated the TextBlob NLP library. It parses unstructured text reviews from patients, calculating mathematical polarity (ranging from -1.0 to 1.0) and subjectivity scores to automatically classify the feedback as Positive, Neutral, or Negative, generating automated actionable suggestions for hospital administrators.

Model Selection & Design Rationale

- **Python as the Orchestrator:** Python was chosen as the central environment due to its unmatched ecosystem. It can run ML models (Scikit-Learn), execute graph math (NetworkX), and bridge to Prolog (PySwip).
- **Expert System (Prolog):** Selected for diagnosis because medical triage relies heavily on strict, interpretable rules (First-Order Logic). Unlike neural networks, a Prolog knowledge base provides complete transparency into *why* a diagnosis was made.

- **Pathfinding (A Search):** While BFS is complete and optimal for unweighted grids, it explores uniformly. A* was selected as the primary routing engine because utilizing a Manhattan Distance heuristic focuses the search directly toward the goal, drastically reducing the number of nodes explored.
- **Predictive Prognosis (Linear Regression):** Selected for predicting recovery days. Since the relationship between age, disease severity, and recovery time is generally linear, Multiple Linear Regression provides a highly interpretable and computationally lightweight baseline.

Application Flow & UI Integration

To make this complex backend usable, I built an interactive web interface using Streamlit. The UI is structured into a 6-step wizard utilizing Streamlit's session_state to pass variables between modules:

1. **Diagnosis:** Takes user demographics and symptoms, Queries the Prolog Expert System.
2. **Emergency Protocol:** Assesses the risk level outputted by Prolog.
3. **Hospital Mapping:** Uses geopy and pydeck to plot the patient's location and nearby simulated hospitals on a 3D scatterplot map.
4. **Routing:** Triggers the A* algorithm and displays a live-tracking progress bar for ambulance dispatch.
5. **Recovery Prediction:** Feeds the patient's age and Prolog risk level into the Scikit-Learn Regression model to predict bed-days.
6. **AI Administration:** Allows administrators to run Haar Cascade Face Detection and process NLP feedback.

Testing & Evaluation Methodology

- **Unit & Integration Testing:** Each Python module was executed independently to ensure internal logic worked before UI integration. Integration testing verified that Python strings (symptoms) correctly translated into Prolog queries via pyswip, and

that Prolog's risk-level output correctly mapped to the integer scale required by the ML model.

- **Search Algorithms:** Evaluated based on execution time and the number of nodes explored. A* consistently out-performed BFS in computational efficiency.
- **Machine Learning:** Evaluated using standard regression metrics (Mean Squared Error and R^2 Score) to ensure prediction accuracy.
- **Edge-Case Testing:** Handled scenarios such as users selecting fewer than 2 symptoms, entering invalid city locations, or webcam access failures with graceful degradation and error messages.

ScreenShots / Result

```
PS C:\Users\Venkata Avinash\Desktop\Smart Hospital Management System> swipl
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.5-14-g4e2eff69c-DIRTY)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

1 ?- ['medical_logic'].
true.

2 ?- best_diagnosis([fever, cough, fatigue], D, S).
D = flu,
S = 3.

3 ?- get_symptoms(flu, S).
S = [fever, cough, fatigue, body_ache].

4 ?- register_patient('Avinash', flu, ID).
ID = "PAT-Avinash-9495".

5 ?- list_patients.
PAT-Avinash-9495 - Avinash - flu
true.
```

Challenges and Problem-Solving

- **Bridging Declarative and Imperative Logic:** Integrating a logical language like Prolog with a procedural language like Python was complex. I utilized the pyswip bridge, but had to write careful parsing logic in app.py to extract variables from the dictionary-like list structures returned by Prolog queries.

- **Handling Ambiguity in Expert Systems:** Early iterations of the Prolog code failed if a patient had symptoms of two different diseases. Implementing the count_matches ranking algorithm allowed the system to handle fuzzy inputs gracefully, acting closer to a real physician's differential diagnosis.
- **State Management:** Because Streamlit reruns the entire Python script upon every button click, preventing the loss of patient data (like the chosen hospital or diagnosis) required implementing robust global session states (st.session_state).

Conclusion and Key Takeaways

The Bring Your Own Project framework provided a profound learning experience. It shifted my perspective from viewing AI algorithms as isolated theoretical concepts to seeing them as interlocking components of a larger software ecosystem.

I learned that Symbolic AI (Prolog) is highly efficient for strict, rule-based medical triage where interpretability is non-negotiable, while Statistical AI (Machine Learning) is better suited for predicting continuous variables like recovery times where rules are inherently blurry. By combining Search Algorithms, Expert Systems, Machine Learning, and UI design, this project serves as a comprehensive culmination of the fundamental principles of Artificial Intelligence.

Future Enhancements

- **Deep Learning for Diagnostics:** Replacing the Prolog symptom-checker with a Convolutional Neural Network (CNN) capable of diagnosing conditions directly from medical imagery (e.g., X-Rays for Tuberculosis).
- **Live Traffic API Integration:** Upgrading the search_engine.py from a simulated grid to utilize the Google Maps Directions API for real-world, traffic-adjusted ambulance routing.
- **Database Integration:** Connecting the patient registry to a persistent SQL database (like PostgreSQL) instead of storing records dynamically in Prolog's volatile runtime memory.

References

1. Russell, S., & Norvig, P. (2009). *Artificial Intelligence: A Modern Approach* (3rd ed.). Prentice Hall.
2. Alpaydin, E. (2016). *Machine Learning: The New AI*. MIT Press.
3. Bratko, I. (2011). *Prolog: Programming for Artificial Intelligence* (4th ed.). Addison-Wesley.
4. Scikit-learn Developers. (n.d.). *Supervised learning: Linear Models*. Scikit-learn Documentation.
5. NetworkX Developers. (n.d.). *Shortest Paths and A* Search*. NetworkX Reference.
6. Streamlit Inc. (n.d.). *Session State Architecture*. Streamlit Documentation.